

## Atelier 2 – Diagnostiquer une conception fragile

1) Les responsabilités mélangées sont : Les interactions avec la base de données et avec l'API, la gestion de notifications, la gestion du système de paiement et la validation d'une commande. **On a une logique métier mélangée à des responsabilités techniques.**

2) Les dépendances fortes sont : La dépendance à la technologie de base de données et sa version, au prestataire de paiement, et au service de notifications, tous en un même module.

**Le principe de Single Responsibility (S de SOLID) n'est donc pas respecté**, car ce module a plusieurs responsabilités qui devraient évoluer indépendamment.

3) Trois abstractions ou interfaces :

- OrderValidation : qui enregistre et valide une commande
- Payment : Gère le paiement
- NotificationService : Gère l'envoi de notifications

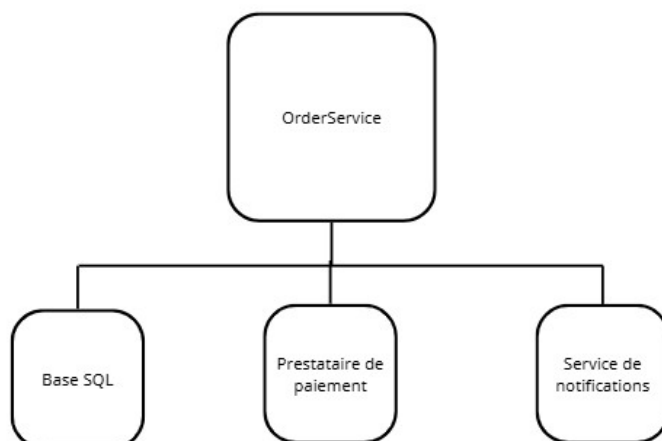
OrderService fera appel à ces 3 éléments pour passer une commande, demander un paiement, envoyer une notification, sans connaître la base de données, le prestataire de paiement, le service de notifications.

4) Il faut séparer la logique métier de la commande et les détails techniques. OrderService doit être responsable de la validation d'une commande, mais ne doit pas connaître directement la base de données, le prestataire de paiement, ou le système de notification. Il devrait faire appel à des éléments séparés qui eux, communiqueraient avec ces éléments externes en utilisant des abstractions ou interfaces. **On a donc une frontière qui sépare logique métier et possibles changements techniques.**

5) passé

6)

**Avant :**



Après :

